

1. What is the difference between environment variables and shell variables?

Environment variables are inherited by all system processes and shells. Shell variables only apply to current shell instances.

2. In Bash, if we run "**export foo=bar**", does it change the environment variable of the current process?

No, it does not. Since "foo" is a shell variable, it does not change the environment variable of current process but sets a value for the child process.

3. The followings are two different ways to print out environment variables. Please describe their differences:

```
$ /usr/bin/env
```

```
$ /usr/bin/strings /proc/$$/environ
```

- a. The program is executed in the child process, so the environment variables of the child process are being printed out.
- b. It prints out the environment variables of current shell process.

4. In our code, when we use **execve()** to execute an external program **xyz**, we pass **NULL** in the third argument. How many environment variables will the process running **xyz** have?

None, since the third argument is the array of the environment variables passed down to xyz process. We pass the NULL value.

5. Bob says that he never uses any environment variable in his code, so he does not need to worry about any security problem caused by environment variables. Is he correct?

No, since the library/external program he uses may use environment variables.

6. A program **abc** invokes an external program **xyz** using **system()**, which is affected by the **PATH** environment variable. When we invoke **abc** from a shell prompt, how does the shell variable **PATH** in the current shell end up affecting the behavior of the **system()** function?

When the shell process executes a command in the child process, PATH is turned into an environment variable. If a shell command is executed in the child process, PATH turns back into shell variable, which affects the behavior of the `system()` function.

7. Please explain why using secure `getenv()` is better than using `getenv()`.

Secure `getenv()` returns a NULL value in case the program is being run in privileged mode, which is used to avoid influence by environment variables.

8. A privileged Set-UID program needs to find out which directory it is currently in. There are two typical approaches. One is to use the `PWD` environment variable, which contains the full path of the current directory. Another approach is to use the `getcwd()` function (you can find its manual online). Please describe which approach you would like to take and why.

The second one is better since it cannot be modified by users and it comes from the system.

9. In Linux, many environment variables are ignored by the dynamic linker if the program to be executed is a Set-UID program. Two such examples are `LD_PRELOAD` and `LD_LIBRARY_PATH`. Please read the manual of `ld-linux` (<https://linux.die.net/man/8/ld-linux>) and explain why the following environment variables are also ignored:

- `LD_AUDIT`
- `LD_DEBUG_OUTPUT`

a. `LD_AUDIT` contains a colon-separated list of use-specified ELF shared objects. If the code is untrusted, it could be dangerous when executed in Set-UID.

b. `LD_DEBUG_OUTPUT` allows the dynamic linker's debugging information to be saved to a file. A malicious user can use this to escalate their permissions.

10. There are two typical approaches for letting normal users do privileged tasks, one is to write a root-owned Set-UID program, and let the user run; another approach is to use a dedicated root daemon to do those privileged tasks for users. Please compare the attack surfaces of these two approaches, and describe which one is more secure.

- Set-UID approach: Normal users have to run a special program to gain root privileges temporally.

- Service Approach: Norma users have to request a privileged service to perform the actions for them.